

Generators são funções especiais do JavaScript.

Elas conseguem:

- pausar a execução
 - continuar depois
 - retornar vários valores aos poucos
-

A ideia principal

Uma função normal retorna apenas UM valor.

Exemplo:

```
function soma() {  
  return 10;  
}
```

Resultado:

```
10
```

Depois do `return`, a função termina.

O problema

E se quisermos:

- retornar vários valores
- um por vez
- sem criar tudo de uma vez?

É aí que entram os Generators.

Criando um Generator

Para transformar uma função em Generator usamos `*`.

```
function* numeros() {  
  
}
```

O `*` significa:

"essa função agora é um Generator"

O yield

Dentro do Generator usamos:

```
yield
```

O `yield` significa:

"retorne esse valor e pause aqui"

Primeiro exemplo

```
function* numeros() {  
  yield 1;  
  yield 2;  
  yield 3;  
}
```

Executando

```
const n = numeros();
```

Aqui acontece algo importante:

A função NÃO executa imediatamente.

Ela cria um objeto Generator.

Usando next()

Para pegar os valores usamos:

```
n.next()
```

Primeira chamada

```
n.next()
```

Resultado:

```
{ value: 1, done: false }
```

O que aconteceu?

O Generator:

1. começou a executar
2. encontrou:

```
yield 1
```

3. retornou 1
 4. pausou naquele ponto
-

Segunda chamada

```
n.next()
```

Resultado:

```
{ value: 2, done: false }
```

Agora ele:

- continua de onde parou
 - encontra `yield 2`
 - pausa novamente
-

Terceira chamada

```
n.next()
```

Resultado:

```
{ value: 3, done: false }
```

Quarta chamada

```
n.next()
```

Resultado:

```
{ value: undefined, done: true }
```

Agora terminou.

Entendendo value e done

value

É o valor retornado.

Exemplo:

```
value: 1
```

done

Indica se terminou.

```
done: false
```

significa:

| "ainda existem valores"

```
done: true
```

significa:

| "acabou"

Visualização mental

Imagine um vídeo pausando:

```
yield 1 → pausa  
yield 2 → pausa  
yield 3 → pausa  
fim
```

Cada `next()`:

| continua o vídeo.

Exemplo visual

```
function* teste() {  
  
  console.log("A");  
  
  yield 1;  
  
  console.log("B");  
  
  yield 2;  
  
  console.log("C");  
  
  yield 3;  
}
```

Executando

```
const t = teste();
```

Primeiro next()

```
t.next();
```

Saída:

```
A
```

Retorno:

```
{ value: 1, done: false }
```

O Generator pausa no primeiro `yield`.

Segundo next()

```
t.next();
```

Saída:

B

Retorno:

```
{ value: 2, done: false }
```

Generator infinito

Generators podem gerar valores infinitamente.

```
function* contador() {  
  let i = 0;  
  while(true) {  
    i++;  
    yield i;  
  }  
}
```

Executando

```
const c = contador();  
  
console.log(c.next());  
console.log(c.next());  
console.log(c.next());
```

Resultado:

```
{ value: 1, done: false }  
{ value: 2, done: false }  
{ value: 3, done: false }
```

Diferença entre return e yield

return	yield
termina a função	pausa a função
retorna uma vez	retorna várias vezes
encerra tudo	continua depois

for...of com Generator

Generators são iteráveis.

Exemplo:

```
function* frutas() {  
  
  yield "Maçã";  
  yield "Banana";  
  yield "Uva";  
  
}  
  
for (const fruta of frutas()) {  
  
  console.log(fruta);  
  
}
```

Resultado:

```
Maçã  
Banana  
Uva
```

Resumo rápido

Conceito	Significado
function*	cria Generator
yield	pausa e retorna valor
next()	continua execução
value	valor retornado
done	informa se terminou

Ideia principal

Generator é:

| uma função que pode pausar e continuar depois.